Software Architecture and Design Specification

Project: Online Quiz System

Version: 1.0

Authors: Team Jackfruit (Tejas, Sanjana, Vikas, Vaishnavi)

Date: 12-10-2026

Status: Baseline for Sprint-1

Revision History

Version	Date	Author	Change Summary
1.0	12-10-2026	Team	First complete version, aligned with SRS v1.1 and the code on `main`

1. Introduction

1.1 Purpose

This document describes the architecture and design of the Online Quiz System, a console-based multiple-choice quiz program written in C. It shows how the requirements in the SRS (v1.1) are met by components, functions and data flow, so that the four team members can build their modules separately and test them against the Test Plan.

1.2 Scope

It covers the whole program: the main menu, the quiz engine, scoring and result display, and result storage with the leaderboard. It does not cover deployment (the program is one local executable) or anything outside the SRS scope, such as accounts, networking or a GUI.

1.3 Audience

Developers building the modules, testers writing test cases, the course evaluators, and anyone who works on the project later.

1.4 Definitions

- **MCQ:** multiple-choice question with exactly one correct option.
- **REQ-n, NFR-n, SEC-n:** functional, non-functional and security requirements in the SRS.
- **STRIDE:** a list of threat types (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege), used in 3.9.

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture (components, pattern, traceability to requirements, security). Section 4 describes the design (sequence diagrams, functions, error handling, screens). Developers should read 3.4 and 4.3 before changing a module. Testers should read 3.8 and 4.4.

2.2 Related documents

SRS (`SRS.md`), Test Plan (`docs/TEST_PLAN.md`), and the team task split with the shared function signatures (`TEAM-TASKS.md`).

3. Architecture

The goals are a small, readable program that four people can build at the same time (NFR-5) and that never crashes on bad input or a missing file (NFR-4). The constraints are plain C (C99) built with `gcc -Wall -std=c99` and no third-party libraries; console only, one user, no network and no database; data is saved only in local text files; and the shared function names and structs in `quiz.h` , `score.h` and `storage.h` are fixed so the modules can be built in parallel.

3.3 Component (UML) Diagram

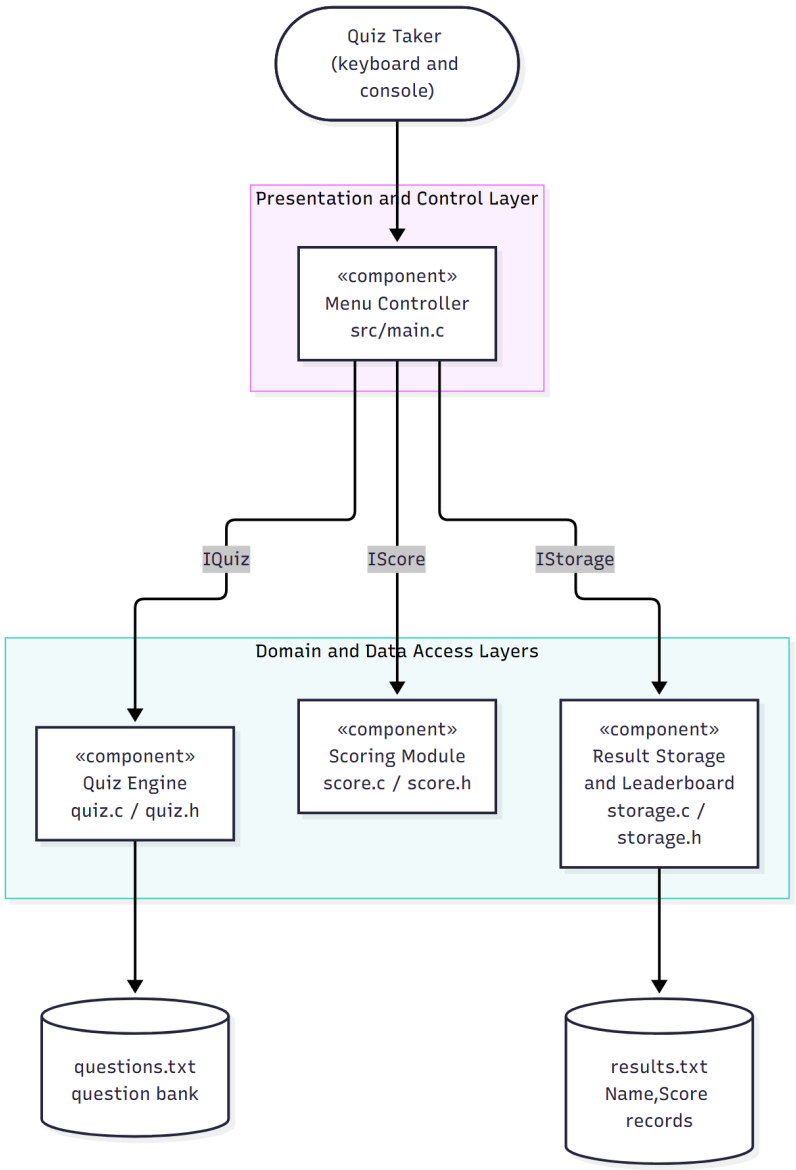


Figure 1: Components, the functions each one offers, and the data files.

3.4 Component Descriptions

Component	Files	What it does	Functions it offers	Owner
Menu Controller	`src/main.c`	Shows the menu, checks the menu choice, asks for the player name, calls the chosen feature, shows instructions, exits cleanly	none (entry point)	Tejas
Quiz Engine	`src/quiz.c`, `include/quiz.h`	Loads the question bank (`data/questions.txt`), shows one question at a time, checks and records each answer	**IQuiz**: `load_questions`, `run_quiz`	Sanjana
Scoring Module	`src/score.c`, `include/score.h`	Compares answers with the correct options, counts correct and incorrect answers, works out the percentage and Pass/Fail (40%), shows the result screen	**IScore**: `calculate_result`, `show_result`	Vikas
Result Storage and Leaderboard	`src/storage.c`, `include/storage.h`	Adds `Name,Score` to `results.txt`, reads the results back, shows the leaderboard from high score to low	**IStorage**: `save_result`, `show_leaderboard`	Vaishnavi

The only link between modules is the two shared data types: `score.h` includes `quiz.h` (for `Question`) and `storage.h` includes `score.h` (for `Result`). Modules do not call each other's functions. The Menu Controller calls them one after another.

3.5 Chosen Architecture Pattern and Rationale

Pattern: layered architecture (one program made of separate modules).

Layer	Contents	Rule
Presentation and control	`main.c`	Talks to the user and calls the layers below; has no scoring or file code
Domain	`quiz.c`, `score.c`	Quiz rules and scoring; knows nothing about the menu
Data access	`storage.c`, the two text files	The only code that reads or writes `results.txt`

Rationale. The program has one user, runs as one process, and has three clear parts, which match how the team shares the work. Layers give each owner their own files (so pull requests do not clash), let each module be tested alone (`tests/test_score.c` uses only `score.c`), and keep file-format details out of the menu code.

Alternatives rejected. Client-server or microservices (there is no network and no second user), one big `main.c` (four people would edit one file and nothing could be tested alone), and a database (the SRS says no database and a text file is enough for a short leaderboard).

Key design decisions

- Text files for questions and results: no extra software is needed and a person can read them.
- Shared function names and structs are fixed, so a module can be built before the modules it depends on exist.
- Fixed-size arrays created by the caller and no dynamic memory, so there are no memory leaks (the question limit is `MAX_QUESTIONS`, 100).

3.6 Technology Stack and Data Stores

- **Language and build:** C (C99), GCC and `make` (`make` builds `quiz`, `make test` runs the score tests).
- **Version control and CI:** Git and GitHub; a GitHub Actions workflow runs `make` on every push and pull request.
- **Question bank:** `data/questions.txt`. For each question: one line of text, four option lines, then one line with the correct letter A to D.
- **Results store:** `results.txt`. One `Name,Score` line per attempt, appended.

3.7 Risks and Mitigations

Risk	Impact	Mitigation
Question file is in the wrong format or missing	Quiz cannot start	`load_questions` returns an error; the menu shows an error and continues (NFR-4)
Two people edit the same file	Merge conflicts	One owner per file; one pull request per issue, reviewed by the repo owner
`results.txt` edited by hand or damaged	Wrong leaderboard	Lines in the wrong format are skipped; the remaining risk is accepted (3.9)

3.8 Traceability to Requirements

Requirement	Component	Function / location
REQ-1 to REQ-6 (menu, validation, navigation, exit)	Menu Controller	`show_menu()`, `read_menu_choice()`, loop and `switch` in `main()`
REQ-7 to REQ-10 (score, counts, percentage, Pass/Fail at 40%)	Scoring	`calculate_result()`, `PASS_PERCENT`
REQ-11 (display result)	Scoring	`show_result()`
REQ-12 (pass result to storage)	Menu Controller, Storage	`save_result(name, &result)` call in `start_quiz()`
REQ-13 (save name and score)	Storage	`save_result()`
REQ-14 to REQ-16 (read, sort, missing file)	Storage	`show_leaderboard()`
REQ-17 (load questions)	Quiz Engine	`load_questions()`
REQ-18 to REQ-22 (show, validate, record, next, finish)	Quiz Engine	`run_quiz()`
NFR-1 Performance	all	no heavy processing; input handled line by line
NFR-2 Security (SEC-1 to SEC-6)	all	see 3.9
NFR-3 Usability	all	`Error:` prefix and re-prompt (4.5)
NFR-4 Reliability	all	error table in 4.4
NFR-5 Maintainability	all	layering and file ownership (3.4, 3.5)

3.9 Security Architecture

The system has no accounts, no network and no special rights. Bad data can come in from two places: the keyboard, which cannot be trusted, and the two local files, which any user of the computer can edit. Spoofing and repudiation are not handled, because there is no login by design (single local user).

Threat	Scenario	Control	Requirement
Tampering	A name with a comma or new line damages the results file or fakes another record	Commas, `\r` and `\n` in the name are replaced by spaces before saving	SEC-2
Tampering	Editing `results.txt` by hand	Risk accepted; lines in the wrong format are skipped	SEC-4
Information disclosure	Error text shows paths or system details	Only fixed one-line messages; no file names or addresses	SEC-1

Threat	Scenario	Control	Requirement
Denial of service, buffer overflow	Very long input, empty input, end of input, missing or unreadable file	Size-limited `fgets` buffers, extra characters discarded, end of input detected, every file call checked	SEC-3, SEC-4, SEC-6
Elevation of privilege	Making the program open a file of the user's choice	File names are fixed in the code and never taken from input	SEC-5

4. Design

4.1 Design Overview

`main()` runs a menu loop. Option 1 calls `start_quiz()`, which asks for a name, loads the questions, runs the quiz, calculates and shows the result, and saves it. Option 2 calls `show_leaderboard()`. Option 3 prints the instructions. Option 4 or end of input leaves the loop and exits.

Shared data structures

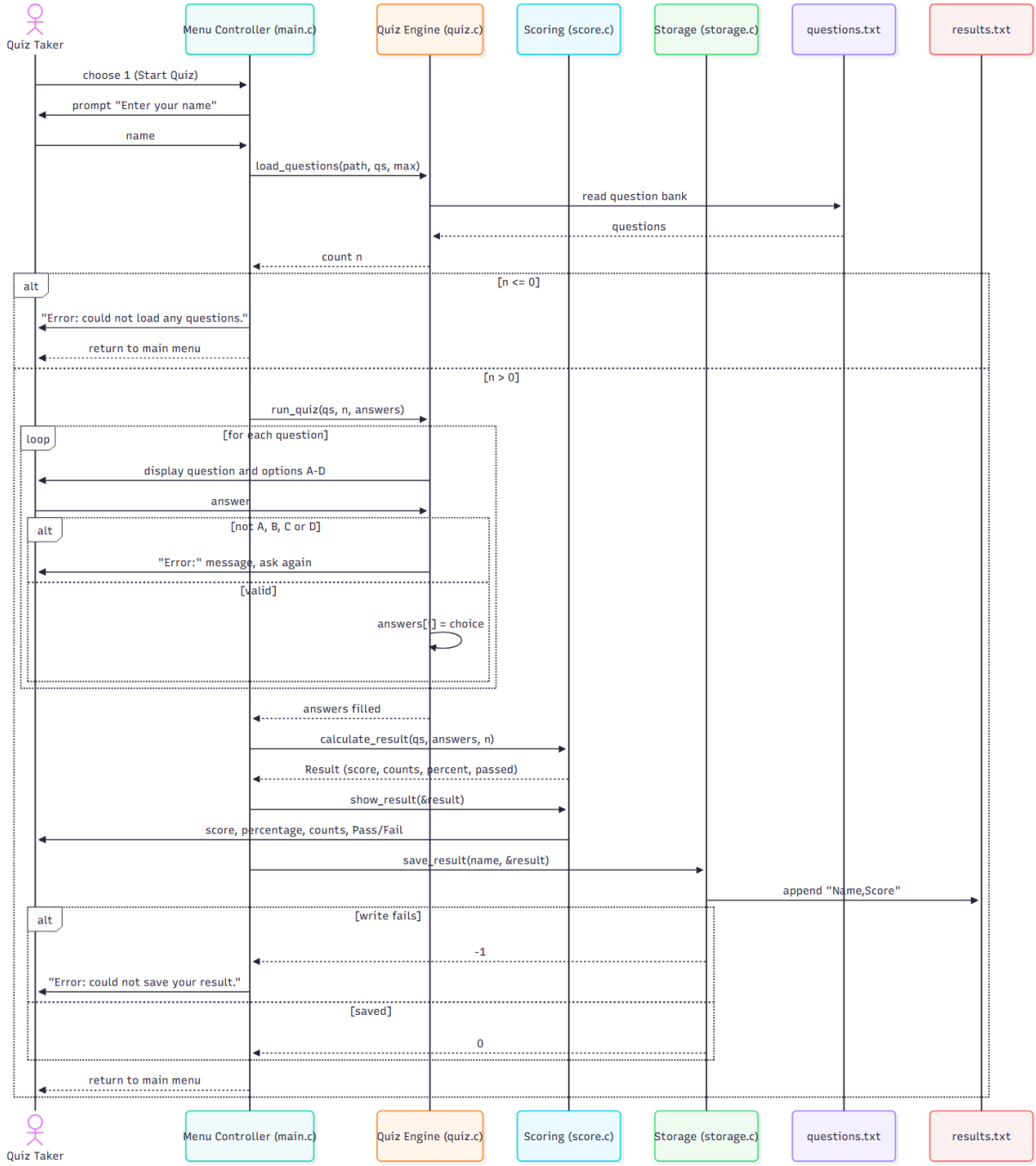
```
typedef struct {            /* quiz.h */
    char text[256];
    char options[4][128];   /* A, B, C, D */
    char correct;           /* 'A'..'D' */
} Question;

typedef struct {            /* score.h */
    int score, correct, incorrect, total;
    double percent;
    int passed;             /* 1 = Pass, 0 = Fail */
} Result;
```

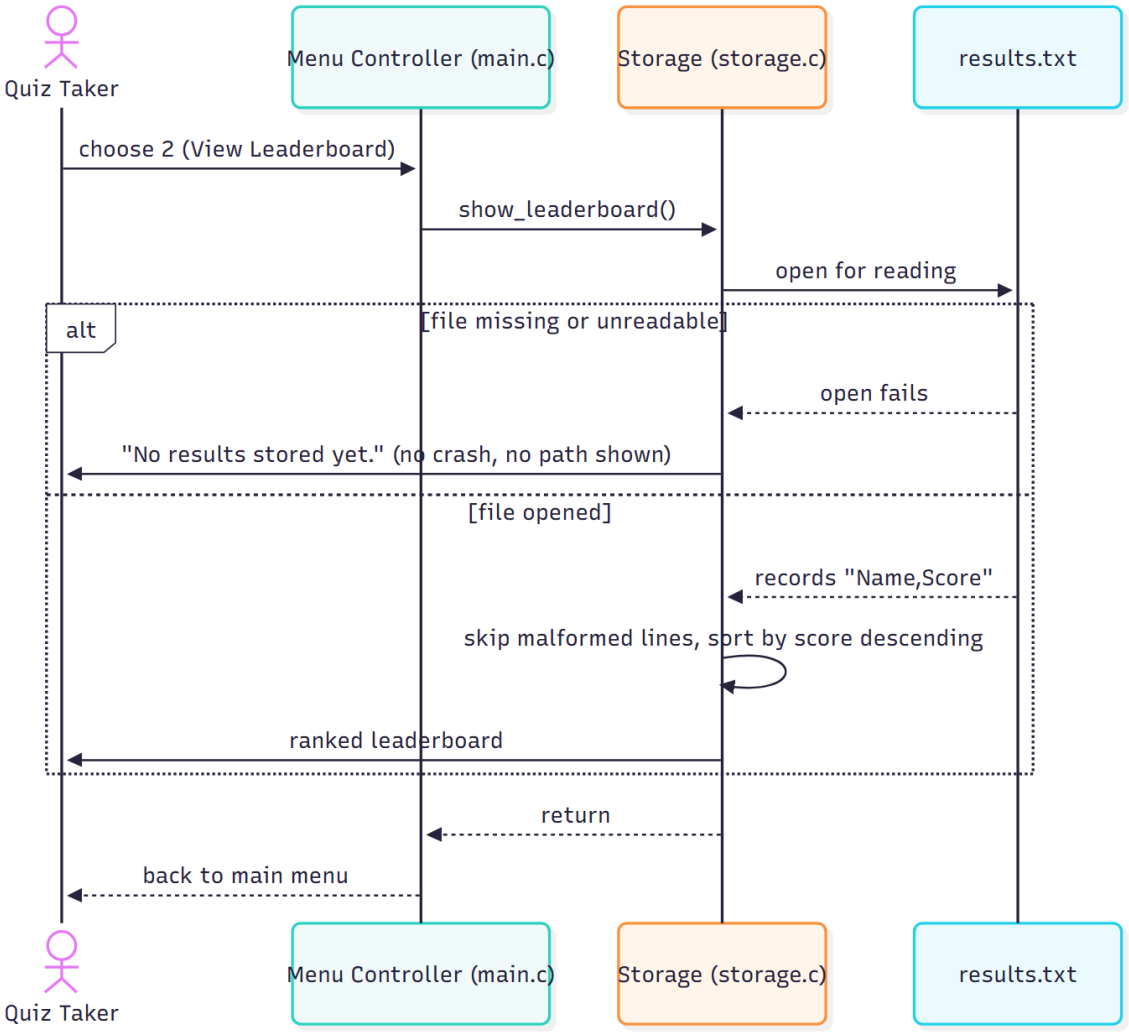
`main.c` holds the questions in a static array of `MAX_QUESTIONS` (100) and the answers in a `char` array that is set to zero first, so unanswered questions never count as correct.

4.2 UML Sequence Diagrams

Sequence 1: Start Quiz (REQ-4, REQ-7 to REQ-13, REQ-17 to REQ-22)



Sequence 2: View Leaderboard (REQ-14 to REQ-16)



4.3 API Design

All interfaces are C functions declared in the header files. There is no network.

Function	Interface	What it does	Returns	On error
`int load_questions(const char *path, Question qs[], int max)`	IQuiz (`quiz.h`)	Reads up to `max` questions from the file into `qs`	Number loaded, or -1 on error	File missing or no valid question: -1 (the caller prints the message)
`void run_quiz(const Question qs[], int n, char answers[])`	IQuiz (`quiz.h`)	Shows questions 0 to n-1 one at a time and stores each answer as `'A'` to `'D'`	nothing; `answers[i]` is filled	Invalid choice: `Error:` line and the same question again. End of input: stops, remaining answers stay 0
`Result calculate_result(const Question qs[], const char answers[], int n)`	IScore (`score.h`)	Checks the answers and works out score, counts, percentage and Pass/Fail (>= 40%)	A `Result`	NULL argument or `n <= 0`: all-zero `Result`
`void show_result(const Result *r)`	IScore (`score.h`)	Prints score, percentage, counts and Pass/Fail	nothing	`r` is NULL: `Error: result is unavailable.`

Function	Interface	What it does	Returns	On error
`int save_result(const char *name, const Result *r)`	IStorage (`storage.h`)	Appends one `Name,Score` line to `results.txt`; commas and new lines in the name become spaces (SEC-2)	0 on success, -1 on error	NULL argument or file problem: -1, prints nothing
`void show_leaderboard(void)`	IStorage (`storage.h`)	Reads `results.txt` and prints the entries from highest to lowest score	nothing	File missing or unreadable: short "no results yet" message; bad lines are skipped

File formats

```
questions.txt   (repeated per question)
<question text>
<option A>
<option B>
<option C>
<option D>
<correct letter A-D>

results.txt     (one per attempt)
<name>,<score>
```

4.4 Error Handling, Logging and Monitoring

Each error is shown as one line starting with `Error:`, and then the program goes back to a safe place (usually the menu). No error message contains a file name, path or system error code (SEC-1).

#	Condition	Response	Requirement
E1	Menu input is not 1 to 4 (letters, empty, too long)	`Error: please enter a number from 1 to 4.` then the menu again	REQ-3, SEC-3
E2	End of input at the menu	Goodbye message, clean exit	SEC-6
E3	Question file missing, unreadable or empty	`Error: could not load any questions.` and back to the menu	NFR-4
E4	Invalid answer during the quiz	`Error:` line and the same question again	REQ-19
E5	Results file cannot be written	`Error: could not save your result.` and back to the menu	SEC-4
E6	Results file missing at the leaderboard, or a line in the wrong format	Short "no results yet" message, or the line is skipped	REQ-16, SEC-4
E7	Scoring called with NULL or `n <= 0`	Zeroed `Result`; `show_result(NULL)` prints `Error: result is unavailable.`	NFR-4

Logging. The quiz keeps no log file, because a log file would be one more place where information could leak. Errors are shown on the screen only.

Monitoring and checks. The GitHub Actions build runs on every push and pull request, and `make test` runs the unit tests for scoring. Both must pass before the repo owner merges a pull request.

4.5 UX Design

- One screen at a time; every screen ends with a prompt.

- Menus are numbered; quiz answers are single letters A to D.
- Every error starts with `Error:` and is followed by a new prompt.
- Optional colour (titles, menu numbers, errors, Pass/Fail, leaderboard top three) using plain ANSI codes. It is switched on only when the output is a terminal, and `NO_COLOR` turns it off, so piped output and the tests stay plain text.

```
===== Online Quiz System =====
1. Start Quiz
2. View Leaderboard
3. Instructions
4. Exit
Enter your choice (1-4):
```

4.6 Open Issues and Next Steps

#	Item	Plan
1	REQ-12 says "score and percentage" are passed to storage, but the file stores name and score only (BR-4)	The whole `Result` is passed; only name and score are saved
2	Equal scores have no set order on the leaderboard	They keep the order they were saved in

5. Appendices

5.1 Glossary

See SRS Appendix A.

5.2 References

- IEEE 42010: Architecture description.
- IEEE 1016: Software design descriptions.
- Online Quiz System SRS v1.1 (`SRS.md`).
- OWASP: input validation guidance; CWE-120 (buffer overflow).

5.3 Tools

Mermaid (component and sequence diagrams), matplotlib (use case diagram), GCC, GNU Make, GitHub Actions.